An introduction to the package MetabolSSMF

Contents

Introduction

This vignette presents the **MetabolSSMF** package, which implements a framework for Simplex-Structured Matrix Factorisation (SSMF) algorithm in R. This package provides a standard algorithm, which allows the users simply to apply the SSMF to perform soft clustering on the dataset that they are interested in. The usage of this package will be illustrated through the example of the application on the simulated metabolomics dataset. This vignette also includes the usage of:

- the gap statistic which guides to select the number of clusters;
- bootstrap resampling approach for estimating confidence intervals;
- soft adjust rand index for compare known and estimated partitions;
- the Shannon diversity index for evaluating the fuzziness of the clustering outputs.

This document gives a quick tour of **MetabolSSMF** functionalities. It was written in R Markdown, using the knitr package for production. See help(package="MetabolSSMF") for further details.

Installation: The latest version of the package can be installed from **Github**:

```
# Install the package
devtools::install_github("WenxuanLiu1996/MetabolSSMF")

# Load the package
library(MetabolSSMF)
```

Data

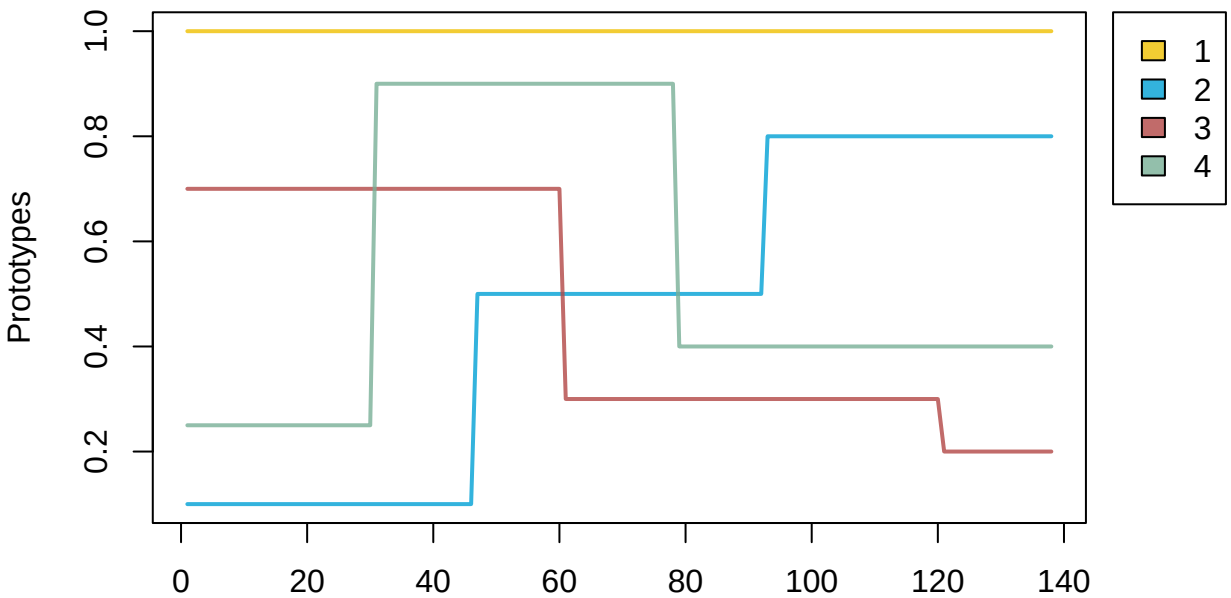
Manually generating a soft memberships matrix H and a prototype matrix W, the simulated dataset is created by the product of H and W. The package includes the simulated data set, simulated prototypes and simulated memberships. They can be loaded through the code `data(SimulatedDataset)`, `data(SimulatedPrototypes)` and `data(SimulatedMemberships)`, respectively.

```
# Simulated data set X
X <- as.matrix(SimulatedDataset)

# Simulated W matrix (prototypes matrix)
W <- as.matrix(SimulatedPrototypes)

# Simulated H matrix (memberships matrix)
H <- as.matrix(SimulatedMemberships)
```

Visualise the simulated data

Visualise the simulated prototypes

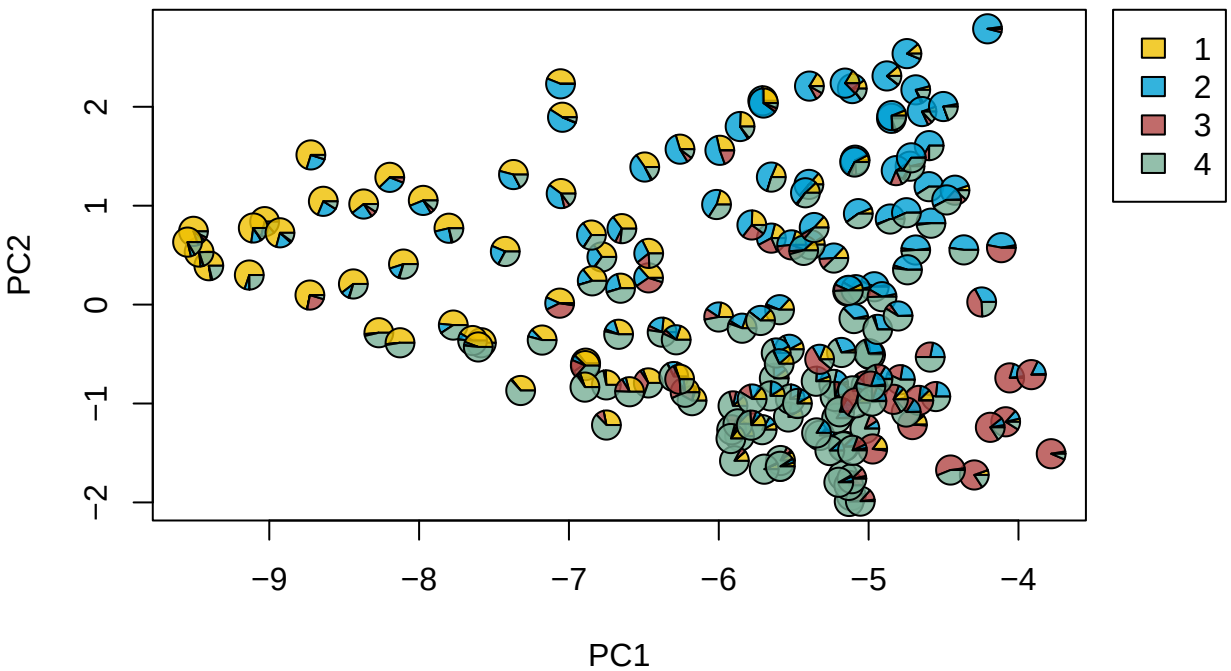
```
# Package for colour palettes
library(ggsci)
# Define 4 different colors
color <- c(pal_jco("default", alpha = 0.8)(7)[2], pal_jama("default", alpha = 0.8)(7)[c(3:5)])

par(mar = c(5.1, 4.1, 4.1, 4.1), xpd = TRUE)
matplot(t(W), type = "l", lty = 1, ylab = "Prototypes", col = color, lwd = 2)
legend("topright", inset = c(-0.15, 0), legend = 1:4, fill = color[1:4])
```



Visualise the observations with simulated soft cluster memberships

The scatter pies plot of 2-dimensional projected clusters. It demonstrates the simulated data with 4 soft clusters, the area of colour in the pies depict the values of soft memberships in the observations

```
# Package for pies plot
library(caroline)
pca_X <- prcomp(X, center = F, scale. = F)
par(mar = c(5.1, 4.1, 4.1, 4.1), xpd = TRUE)
pies(lapply(apply(H, 1, list), unlist), color.table = nv(color[1:4], paste0("Pty",
    1:4)), x0 = pca_X$x[, 1], y0 = pca_X$x[, 2], radii = 2, ylab = "PC2", xlab = "PC1")
legend("topright", inset = c(-0.15, 0), legend = 1:4, fill = color[1:4])
```



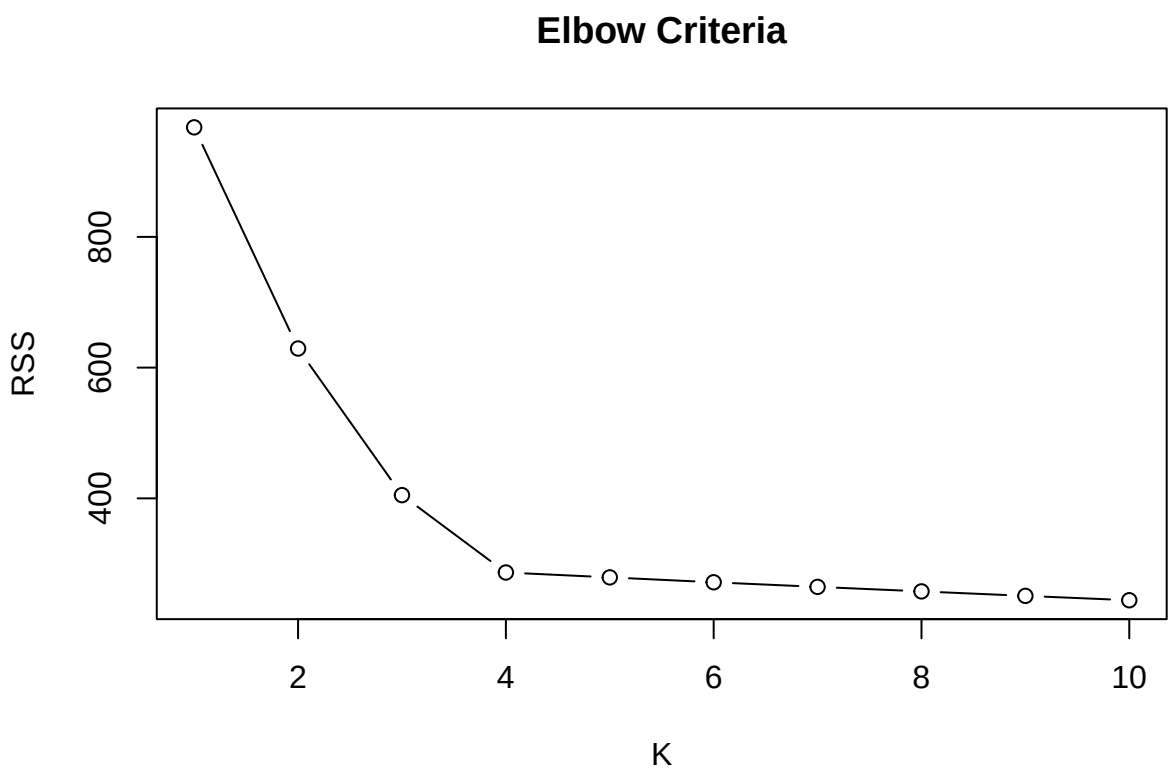
Choose the number of clusters

The algorithm can be run with only one k every time. Running $k(=1,\ldots,K)$ times SSMF algorithm using `for` loop and saving the results to a list are good for processing 'elbow criteria' or the gap statistics in the next step.

```
# Set the maximum k that is used in loop.
K <- 10

# Run the SSMF algorithm with various k and save the results as a list
fit_SSMF <- list()
for (k in 1:K) {
    fit <- ssmf(X, k = k, lr = 0.001, meth = "kmeans")
    fit_SSMF[[k]] <- fit
}
```

Plot the residual sum of squares ('"elbow criteria")

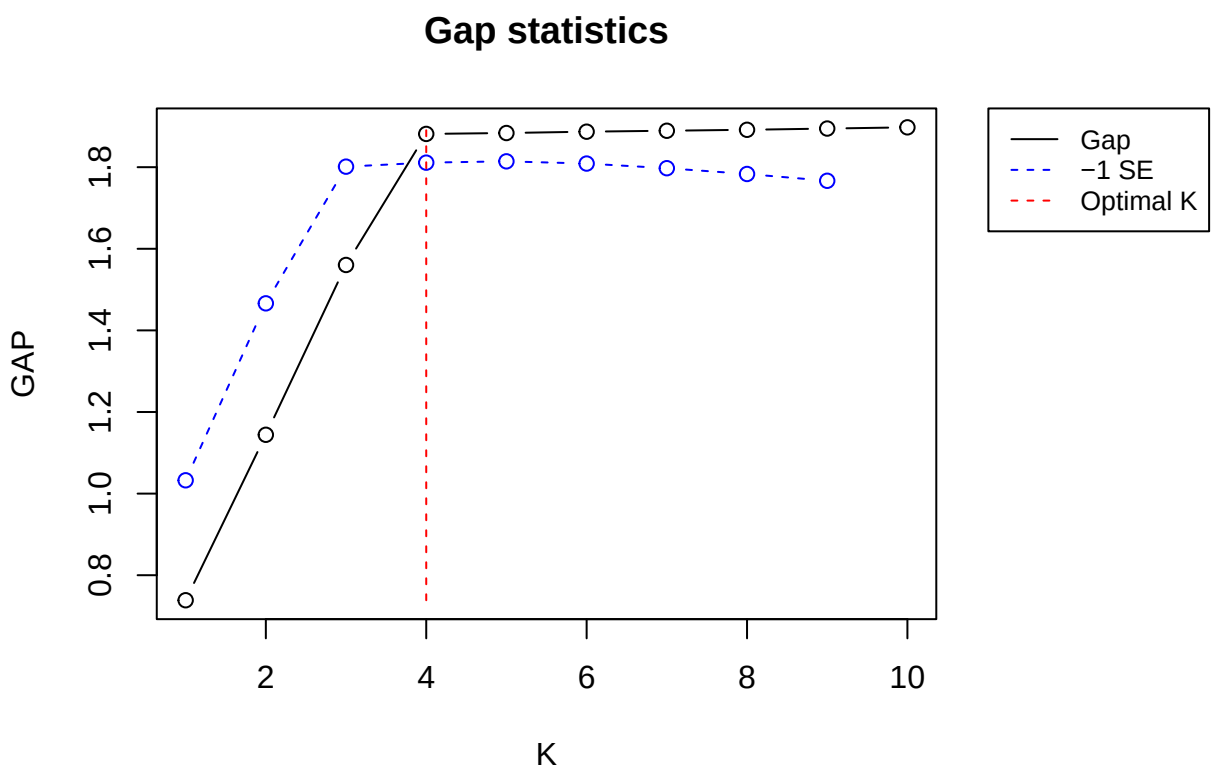
```
# Extract the RSS and save as a vector
rss_SSMF <- unlist(lapply(fit_SSMF, function(x) x$SSE))

# Plot RSS
plot(1:K, rss_SSMF, type = "b", xlab = "K", ylab = "RSS", main = "Elbow Criteria")
```



If the 'elbow' is not clear, the gap statistic is suggest.

Gap statistic

```
# Apply the gap statistics to the simulated data
fit_gap <- gap(X, rss = rss_SSMF)

# Visualise the results
par(mar = c(5.1, 4.1, 4.1, 8.1), xpd = TRUE)
plot(x = c(1:K), y = fit_gap$gap, type = "b", ylim = range((fit_gap$gap), (fit_gap$gap -
    fit_gap$standard.error)[-1]), xlab = "K", ylab = "GAP", main = "Gap statistics")
lines(x = c(1:(K - 1)), y = (fit_gap$gap - fit_gap$standard.error)[-1], col = "blue",
    lty = 2, type = "b")
lines(x = rep(fit_gap$optimal.k, 2), y = range(fit_gap$gap), lty = 2, col = "red")
legend("topright", inset = c(-0.35, 0), col = c("black", "blue", "red"), lty = c(1,
    2, 2), legend = c("Gap", "-1 SE", "Optimal K"), cex = 0.8)
```



Based on the results of the 'elbow criteria' and gap statistics, both of them suggest the optimal number of clusters/prototypes is 4.

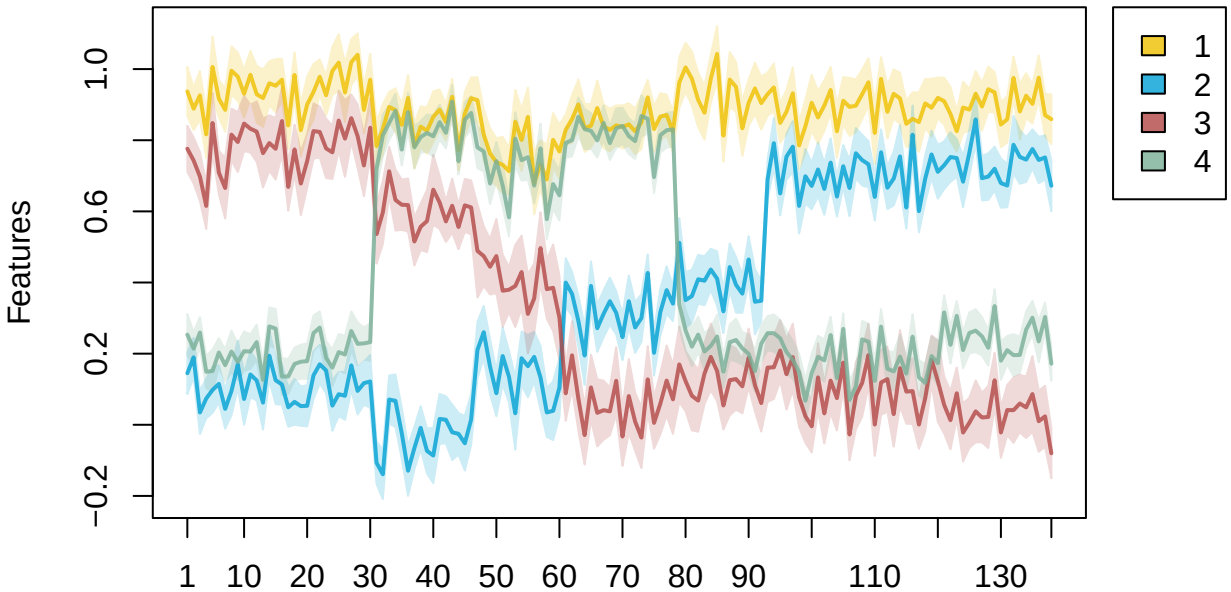
Bootstrap

```
# Set the number of times to bootstrap
M <- 1000

# Bootstrap Initial H (memberships) matrix to start the algorithm Based on the
# results above, k=4 here
initialisation <- init(data = X, k = 4, method = "kmeans")
fit_boot <- bootstrap(data = X, k = 4, H = initialisation$H, mtimes = M)
```

Visualise the bootstrap estimation and confidence intervals

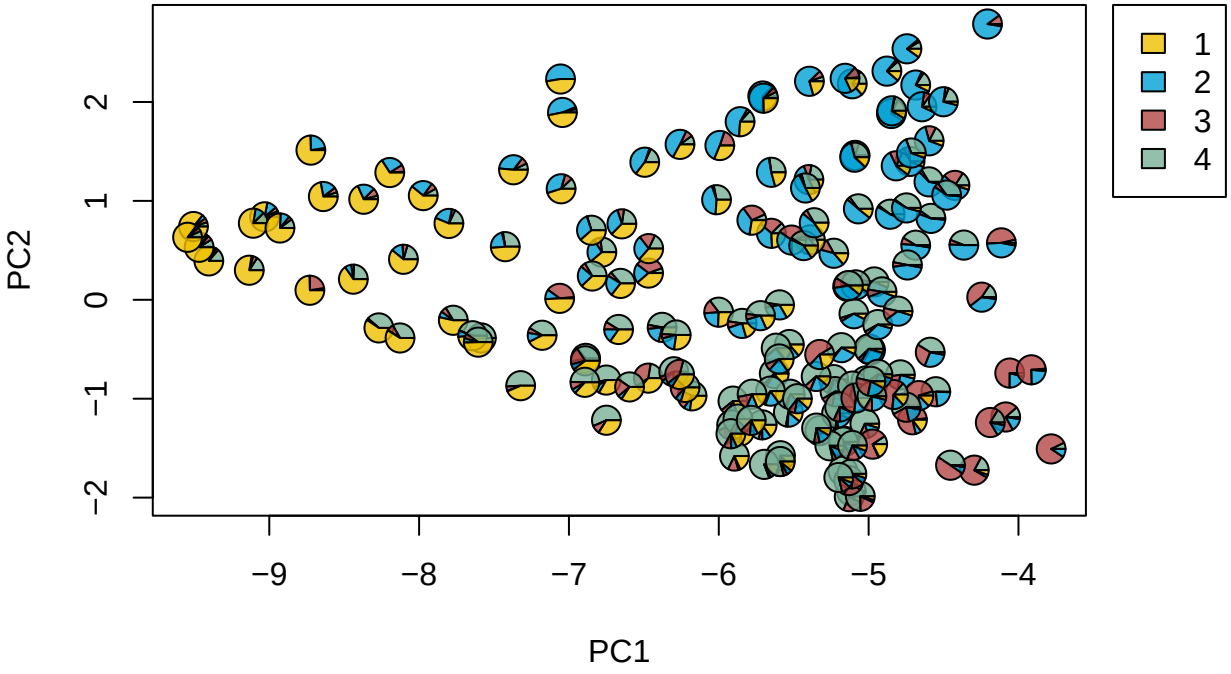
```
par(mar = c(5.1, 4.1, 4.1, 4.1), xpd = TRUE)
matplot(t(fit_boot$W.est[c(2, 4, 1, 3), ]), type = "n", xaxt = "n", ylab = "Features",
    ylim = range(fit_boot$lower, fit_boot$upper))
for (i in 1:4) {
    alpha <- 0.2
    plat <- c(pal_jco("default", alpha = alpha)(7)[2], pal_jama("default", alpha = alpha)(7)[c(3:5)])
    col <- switch(i, plat[1], plat[2], plat[3], plat[4])
    polygon(c(1:138, 138:1), c(fit_boot$upper[c(2, 4, 1, 3), ][i, ], rev(fit_boot$lower[c(2,
        4, 1, 3), ][i, ])), col = col, border = col, lty = 1)
}
matplot(t(fit_boot$W.est[c(2, 4, 1, 3), ]), type = "l", lty = 1, ylab = "Features",
    add = T, col = color, lwd = 2)
```

```r
legend("topright", inset = c(-0.15, 0), legend = 1:4, fill = color[1:4])
axis(1, at = c(1, seq(10, 130, 10), 138), labels = c(1, seq(10, 130, 10), 138))
```



Visulise the observations with estimated soft cluster memberships

```r
par(mar = c(5.1, 4.1, 4.1, 4.1), xpd = TRUE)
pies(lapply(apply(fit_SSMF[[4]]$H, 1, list), unlist), color.table = nv(color[1:4],
    c(4, 3, 2, 1)), x0 = pca_X$x[, 1], y0 = pca_X$x[, 2], radii = 2, ylab = "PC2",
    xlab = "PC1")
legend("topright", inset = c(-0.15, 0), legend = 1:4, fill = color[1:4])
```



Soft Adjusted Rand Index (sARI)

A soft adjusted Rand index (sARI) is proposed to compare the two soft partitions (e.g., the true and estimated fuzzy memberships).

```
# Compare the true and estimated fuzzy memberships
sARI(fit_SSMF[[4]]$H, H)
```

```
## [1] 0.3439103
```

```
# Self comparison of the true fuzzy memberships
sARI(H, H)
```

```
## [1] 0.3816684
```

Shannon diversity index

Shannon diversity index, an information theory is introduced to evaluate the average uncertainty in the soft clustering partition.

```
E <- rep(NA, nrow(fit_SSMF[[4]]$H))
for (i in 1:nrow(fit_SSMF[[4]]$H)) {
    E[i] <- diversity(fit_SSMF[[4]]$H[i, ], two.power = T)
}
mean(E)
```

```
## [1] 2.709743
```

The estimated soft memberships of the simulated observations have an average 2 to the power of Shannon diversity index 2.71, indicating that most observations softly clustered by SSMF are fuzzy in 2 clusters.

Addition NOTE

The SSMF algorithm has randomness, this document presents one of the situation with a certain seed. The results might be various with different seeds that set in R. It is suggest to remove the seed and run the algorithm multiple times with a certain value of k (the number of clusters/prototypes). Then, select a result with the lowest RSS, save it to the list and continue processing next steps.